

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA		
CO1 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	Shaik Nuha	Reg. No.:	192472242
Section / Batch:	CSE-AI	Date:	

Code of Conduct

I Shaik Nuha (192472242) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____ Sk. Nuha _____.

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Information Extraction	Regex-Based Information Extraction	1
Pattern Matching	Regex Pattern Matching	2
Regular Expression Patterns	Regex-Based Input Validation	3

Section 1: Regular Expressions – Resume Information Extraction

1. A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.

Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:

Extract the candidate's name.

Identify email addresses.

Extract mobile numbers.

Detect technical skills (Python, Java, SQL, Machine Learning, NLP).

Extract years of experience.

Generate a structured summary of the candidate's profile.

Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

Section 2: Regular Expressions – Product Search System

2. An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

Design and implement a Python-based product search system using Regular Expressions that performs the following tasks:

Search products using an exact keyword.

Perform prefix-based searches.

Perform suffix-based searches.

Search using partial keywords.

Perform case-insensitive searches.

Display all matching product names.

Generate a report showing the total number of matching products for each search.

Section 3: Regular Expressions – University Registration System

A university is developing an automated online registration portal to verify student information before course enrollment.

Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:

Validate the student register number.

Validate the institutional email address.

Validate the course code.

Validate the semester information.

Validate the student's mobile number.

Display appropriate validation messages for each field.

Generate a final registration status report indicating whether the registration is successful.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Q. No. / Criteria	Understanding of Problem Context	Application of Concepts	Problem-Solving Approach	Accuracy of Solution	Clarity	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1. Resume Information Extraction System using Regular Expressions

Aim

To design and implement a Python-based Resume Information Extraction System using Regular Expressions that automatically extracts important candidate information from resumes, generates a structured summary, and identifies eligible candidates based on predefined criteria.

Problem Statement

A multinational recruitment company receives thousands of resumes every day in text format. Manually reviewing these resumes is time-consuming and error-prone. Therefore, the HR department requires an automated system that can extract candidate information using Regular Expressions.

The system should:

- Extract the candidate's name.
- Identify email addresses.
- Extract mobile numbers.
- Detect technical skills (Python, Java, SQL, Machine Learning, NLP).
- Extract years of experience.
- Generate a structured summary of the candidate's profile.
- Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

Objectives

- Automate resume processing.
- Reduce manual effort in recruitment.
- Improve the accuracy of information extraction.
- Identify eligible candidates quickly.
- Generate a structured candidate profile.

Requirements:

Software Requirements

- Python 3.x
- IDLE / VS Code / Jupyter Notebook

Python Module

- re (Regular Expressions)

Algorithm

1. Import the re module.
2. Read the resume text.
3. Use Regular Expressions to extract:
 - Candidate Name
 - Email Address

- Mobile Number
 - Technical Skills
 - Years of Experience
4. Display the extracted information.
 5. Check whether:
 - Experience is at least 2 years.
 - Python skill is present.
 6. If both conditions are satisfied, display Eligible Candidate.
 7. Otherwise display Not Eligible.

Python Program

```
import re
resume = """
Name: Rahul Sharma
Email: rahul.sharma@gmail.com
Mobile: 9876543210
Skills: Python, Java, SQL, Machine Learning, NLP
Experience: 3 years
"""

name = re.search(r"Name:\s*(.*)", resume)
print("Candidate Name:", name.group(1))
emails = re.findall(r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}", resume)
print("Email:", emails)
mobile = re.findall(r"\b[6-9]\d{9}\b", resume)
print("Mobile:", mobile)
skills = ["Python", "Java", "SQL", "Machine Learning", "NLP"]
found_skills = []
for skill in skills:
    if re.search(skill, resume, re.IGNORECASE):
        found_skills.append(skill)
print("Skills:", found_skills)
experience = re.search(r"(\d+)\s+years", resume)
exp = int(experience.group(1))
print("Experience:", exp, "years")
print("\nCandidate Summary")
print("-----")
print("Name:", name.group(1))
print("Email:", emails[0])
print("Mobile:", mobile[0])
print("Skills:", ", ".join(found_skills))
print("Experience:", exp, "years")
if exp >= 2 and "Python" in found_skills:
    print("\nStatus: Eligible for Shortlisting")
else:
    print("\nStatus: Not Eligible")
```

Sample Output

```
Candidate Name: Rahul Sharma
Email: ['rahul.sharma@gmail.com']
Mobile: ['9876543210']
Skills: ['Python', 'Java', 'SQL', 'Machine Learning', 'NLP']
Experience: 3 years
```

Candidate Summary

```
-----
Name: Rahul Sharma
Email: rahul.sharma@gmail.com
Mobile: 9876543210
Skills: Python, Java, SQL, Machine Learning, NLP
Experience: 3 years
```

```
Status: Eligible for Shortlisting
```

Explanation

The program uses the Python re module to identify different patterns in the resume text. Regular Expressions are used to extract the candidate's name, email, mobile number, technical skills, and years of experience. After extracting the information, the program checks whether the candidate has at least two years of experience and possesses Python skills. If both conditions are satisfied, the candidate is considered eligible for shortlisting.

Advantages

- Fast resume processing.
- Reduces manual work.
- Accurate extraction of candidate details.
- Easy to modify for additional skills.
- Improves recruitment efficiency.

Applications

- Recruitment companies.
- HR departments.
- Job portals.
- Resume screening systems.
- Placement management systems.

Result

The Resume Information Extraction System was successfully implemented using Python Regular Expressions. The system correctly extracted candidate information, generated a structured profile summary, and identified eligible candidates according to the specified criteria.

2.Product Search System using Regular Expressions

Aim

To design and implement a Python-based Product Search System using Regular Expressions that performs flexible product searches using exact keywords, prefix, suffix, partial keywords, and case-insensitive matching.

Problem Statement

An e-commerce company plans to improve its product search engine to enhance customer experience through flexible keyword matching. The system should:

- Search products using an exact keyword.
- Perform prefix-based searches.
- Perform suffix-based searches.
- Search using partial keywords.
- Perform case-insensitive searches.
- Display all matching product names.
- Generate a report showing the total number of matching products for each search.

Objectives

- Improve product searching.
- Reduce search time.
- Support multiple search methods.
- Display all matching products.
- Count the total number of matches.

Software Requirements

- Python 3.x
- IDLE / VS Code
- re module

Algorithm

1. Import the re module.
2. Store product names in a list.
3. Display the search menu.
4. Accept the user's choice.
5. Perform the selected search using Regular Expressions.
6. Display matching products.
7. Display the total number of matches.

Python Program

```
import re
products = [
    "Laptop",
    "Laptop Bag",
    "Mouse",
    "Keyboard",
    "Smartphone",
    "Smart Watch",
    "Phone Case",
    "Bluetooth Speaker",
    "Python Book",
    "Java Book"
]
keyword = input("Enter search keyword: ")
print("\nExact Match")
exact = [p for p in products if re.fullmatch(keyword, p, re.IGNORECASE)]
print(exact)
print("\nPrefix Match")
prefix = [p for p in products if re.match(keyword, p, re.IGNORECASE)]
print(prefix)
print("\nSuffix Match")
suffix = [p for p in products if re.search(keyword + r"$", p, re.IGNORECASE)]
print(suffix)
print("\nPartial Match")
partial = [p for p in products if re.search(keyword, p, re.IGNORECASE)]
print(partial)
print("\nCase Insensitive Match")
case = [p for p in products if re.search(keyword, p, re.IGNORECASE)]
print(case)
print("\nSearch Report")
print("Exact Matches:", len(exact))
print("Prefix Matches:", len(prefix))
print("Suffix Matches:", len(suffix))
print("Partial Matches:", len(partial))
```

Sample Output

```
Enter search keyword: n

Exact Match
[]

Prefix Match
[]

Suffix Match
['Smart Watch']

Partial Match
['Smartphone', 'Smart Watch', 'Phone Case', 'Bluetooth Speaker', 'Python Book']

Case Insensitive Match
['Smartphone', 'Smart Watch', 'Phone Case', 'Bluetooth Speaker', 'Python Book']

Search Report
Exact Matches: 0
Prefix Matches: 0
Suffix Matches: 1
Partial Matches: 5
|
```

Explanation

The program stores product names in a list and performs different types of searches using Regular Expressions. It supports exact, prefix, suffix, partial, and case-insensitive searches and displays all matching products along with the total count.

Advantages

- Fast searching.
- Flexible keyword matching.
- User-friendly.
- Reduces search time.
- Easy to extend.

Applications

- Online Shopping Websites
- E-commerce Platforms
- Product Catalog Systems
- Inventory Management

Result

The Product Search System was successfully implemented using Python Regular Expressions and correctly displayed matching products for different search methods.

3. University Registration Validation System using Regular Expressions

Aim

To design and implement a Python-based validation system using Regular Expressions to verify student registration details before course enrollment.

Problem Statement

A university is developing an automated online registration portal to verify student information before course enrollment. The system should:

- Validate the student register number.
- Validate the institutional email address.
- Validate the course code.
- Validate the semester information.
- Validate the student's mobile number.
- Display appropriate validation messages.
- Generate the final registration status report.

Objectives

- Validate student details.
- Prevent incorrect registrations.
- Ensure correct data entry.
- Display validation messages.

Software Requirements

- Python 3.x
- IDLE / VS Code
- re module

Algorithm

1. Import the re module.
2. Accept student details.
3. Validate:
 - Register Number
 - Institutional Email
 - Course Code
 - Semester
 - Mobile Number
4. Display validation messages.
5. If all validations are successful, display Registration Successful.
6. Otherwise, display Registration Failed.

Python Program

```
import re
register = input("Enter Register Number: ")
email = input("Enter Institutional Email: ")
course = input("Enter Course Code: ")
semester = input("Enter Semester: ")
mobile = input("Enter Mobile Number: ")
status = True
if re.fullmatch(r"\d{9}", register):
    print("Register Number: Valid")
else:
    print("Register Number: Invalid")
    status = False
if re.fullmatch(r"[A-Za-z0-9._%+-]+@saveetha\.com", email):
    print("Email: Valid")
else:
    print("Email: Invalid")
    status = False
if re.fullmatch(r"[A-Z]{2,4}\d{4}", course):
    print("Course Code: Valid")
else:
    print("Course Code: Invalid")
    status = False
if re.fullmatch(r"[1-8]", semester):
    print("Semester: Valid")
else:
    print("Semester: Invalid")
    status = False
if re.fullmatch(r"[6-9]\d{9}", mobile):
    print("Mobile Number: Valid")
else:
    print("Mobile Number: Invalid")
    status = False
print("\nRegistration Report")
print("-----")
if status:
    print("Registration Successful")
else:
    print("Registration Failed")
```

Sample Output

```
Enter Register Number: 192472242
Enter Institutional Email: 192472242.simats@saveetha.com
Enter Course Code: DSA0307
Enter Semester: 5
Enter Mobile Number: 9490830791
Register Number: Valid
Email: Valid
Course Code: Valid
Semester: Valid
Mobile Number: Valid

Registration Report
-----
Registration Successful
|
```

Explanation

The program validates each student detail using Regular Expressions. If all fields match the required format, the registration is successful; otherwise, the system displays appropriate validation messages and reports that registration has failed.

Advantages

- Ensures accurate student data.
- Reduces manual verification.
- Prevents invalid registrations.
- Easy to maintain.
- Fast validation.

Applications

- University Registration Portals
- Student Information Systems
- College Admission Systems
- Online Enrollment Platforms

Result

The University Registration Validation System was successfully implemented using Python Regular Expressions. It correctly validated all input fields and generated the final registration status based on the validation results.